

UNIVERSITÉ DE YAOUNDÉ I

Faculté des Sciences

Département d'Informatique



Master I Systèmes et Réseaux

TRAVAIL PRATIQUE — INF4218

PROGRAMMATION DISTRIBUÉE

SYSTÈME DISTRIBUÉ D'APPRENTISSAGE FÉDÉRÉ

MEMBRES DU GROUPE

NOM ET PRÉNOMS	MATRICULE
MFENJOU ANAS CHERIF	21T2330
EDU GUIEDI HERMANN ARNOLD	22T2876
LANGOUL FADILA MARIAMA MOUNIRA	21T2528

ANNÉE ACADÉMIQUE

2025–2026

SUPERVISÉ PAR

Dr ADAMOU HAMZA

1. Introduction

1.1 Contexte et objectifs

Ce rapport présente le travail pratique réalisé dans le cadre du cours **INF4218 — Programmation Distribuée** (Master I Systèmes et Réseaux, Université de Yaoundé I). Il s'appuie sur les fondements des systèmes distribués exposés par Van Steen et Tanenbaum (*Distributed Systems*, chapitres 1 à 8).

L'objectif est de concevoir et d'implémenter un **système distribué d'apprentissage fédéré** (*Federated Learning*) : plusieurs clients entraînent un modèle localement sur des données privées, transmettent leurs gradients à un serveur central qui les agrège pour produire un modèle global. Ce scénario permet d'illustrer concrètement la coordination distribuée, la communication RPC, la cohérence des événements et la tolérance aux pannes.

Le code source, les tests et les scripts de démonstration sont disponibles sur le dépôt GitHub : <https://github.com/CherifMfenjou/INF4218-TPSD>.

1.2 Concepts obligatoires

Concept	Chapitre Tanenbaum	Module implémenté
Nommage plat	6.2	<code>src/naming/flat.py</code>
Nommage structuré	6.3	<code>src/naming/structured.py</code>
Nommage par attributs	6.4	<code>src/naming/attribute.py</code>
Horloges logiques de Lamport	5.2.1	<code>src/coordination/lamport.py</code>
Élection de leader (Bully)	5.4.1	<code>src/coordination/bully.py</code>
Exclusion mutuelle distribuée	5.3.3	<code>src/coordination/mutual_exclusion.py</code>
Communication RPC/gRPC	4	<code>proto/federation.proto</code>
Tolérance aux pannes (checkpoint)	8.6.2	<code>src/fault_tolerance/checkpoint.py</code>

1.3 Technologies et livrables

Stack : Python 3.14, gRPC/Protocol Buffers, NumPy (simulation des gradients), pytest.

Livrables : architecture client-serveur à trois niveaux ; service gRPC (`FederationCoordinator`, 12 RPC) ; 38 tests unitaires + 1 test d'intégration (tous passent) ; script `./run_demo.sh` ; benchmarks reproductibles (`experiments/benchmarks.py`).

2. Conception

2.1 Architecture globale

Le système suit une architecture en trois couches :

```
+-----+
| COUCHE APPLICATION                                |
| Rounds FL, entraînement local, agrégation des gradients |
+-----+
| COUCHE MIDDLEWARE                                |
| Lamport, Bully, mutex, nommage (3 types), checkpoint |
+-----+
| COUCHE RÉSEAU (gRPC)                                |
| RPC unary + streaming (gradients et modèle global)    |
+-----+
```

Serveur coordinateur (`FederationCoordinator`, process 0) : coordonne les rounds, maintient le modèle global, gère les registres de nommage, applique Lamport/Bully/mutex et effectue les checkpoints.

Clients fédérés (`FederatedClient`) : s'enregistrent via les trois nommages, entraînent localement, échangent gradients et modèle, envoient des heartbeats horodatés.

2.2 Flux d'un round

Client	Coordinateur
---- Heartbeat (Lamport ts) ---->	
<--- Round actuel + ts -----	
[Entraînement local]	
---- UploadGradients (gRPC) ---->	[Mutex : agrégation]
<--- DownloadModel (streaming) ----	

```
| [Checkpoint si round % 3 == 0] |
```

Chaque round associe un signal de vie, un calcul local de gradients, un transfert vers le serveur, une agrégation protégée par mutex et une diffusion du modèle global mis à jour.

2.3 Systèmes de nommage

Plat (UUID) : identifiant opaque unique par client ; résolution $O(1)$ dans un registre central ; clé de liaison pour gradients, heartbeats et checkpoints.

Structuré : chemins hiérarchiques (`/federation/clients/client1/models/local`), inspirés du DNS et des systèmes de fichiers Unix.

Par attributs : recherche par paires clé-valeur (`dataset: iris, location: local`) via `QueryByAttributes`, permettant une sélection dynamique des participants.

Les trois enregistrements sont effectués simultanément lors de l'appel `RegisterClient`.

2.4 Mécanismes de coordination

Lamport : chaque processus maintient un compteur C_i . Événement local : $C_i \leftarrow C_i + 1$. Réception d'un message de timestamp ts : $C_j \leftarrow \max(C_j, ts) + 1$. Si $a \rightarrow b$ (*happens-before*), alors $C(a) < C(b)$.

Bully : le processus d'ID maximal vivant devient coordinateur. En cas de panne, envoi de `ELECTION` aux IDs supérieurs ; le gagnant annonce `COORDINATOR`.

Ricart-Agrawala : l'agrégation constitue une section critique protégée par mutex distribué. Le plus petit timestamp Lamport accède en premier ; absence de deadlock et de famine.

Checkpoint : sauvegarde JSON tous les 3 rounds (round, poids, horloge Lamport, ID coordinateur). Recovery automatique au redémarrage via `_recover_state()`.

2.5 Contribution des concepts au système fédéré

Concept	Rôle dans le FL	Apport	Sans ce concept
Architecture 3 niveaux	Sépare entraînement, coordination, transport	Modules testables et évolutifs	Monolithe fragile

Concept	Rôle dans le FL	Apport	Sans ce concept
Nommage plat	UUID client (<code>flat_id</code>)	Identité unique indépendante de l'IP	Collisions, écrasement de gradients
Nommage structuré	Chemin hiérarchique	Annuaire administrable (type DNS)	Registre ingérable
Nommage par attributs	<code>{dataset, location}</code>	Sélection dynamique des clients	Liste statique de participants
Lamport	Timestamp dans chaque message gRPC	Ordre causal des rounds ; base du mutex	Incohérence du modèle global
Mutex Ricart-Agrawala	Protège <code>aggregate_round()</code>	Intégrité de <code>model_weights</code>	Corruption concurrente
Bully	Élection du coordinateur	Continuité de service après panne	Arrêt total du système
gRPC	Canal client-serveur typé	Unary + streaming interopérable	Formats ad hoc
Checkpoint	Sauvegarde périodique	Reprise rapide ; perte max. 2 rounds	Perte totale du progrès

L'architecture isole la couche réseau (gRPC) de la coordination (Lamport, Bully, mutex) : la communication peut évoluer sans modifier l'algorithme d'agrégation. Le nommage triple garantit l'identification, la navigation et la sélection des clients. Lamport et le mutex assurent respectivement l'ordre causal et l'intégrité de l'agrégation ; Bully et le checkpoint garantissent la continuité et la persistance du service.

3. Implémentation

3.1 Structure du projet

TPSD/

```
proto/federation.proto      -- Service gRPC (12 RPC)
src/naming/                  -- flat, structured, attribute
src/coordination/           -- lamport, bully, mutual_exclusion
src/fault_tolerance/        -- checkpoint, recovery
src/server/coordinator.py   -- Serveur coordinateur
src/client/federated_client.py
tests/                       -- 38 tests unitaires
experiments/benchmarks.py   -- 4 expériences
run_demo.sh                  -- Démonstration reproductible
```

3.2 Communication gRPC

Le service FederationCoordinator expose 12 RPC définis dans `federation.proto` :

- **Unary** : RegisterClient, Heartbeat, RequestMutualExclusion, ReleaseMutualExclusion, ResolveName, QueryByAttributes, SendElection, AnnounceCoordinator.
- **Streaming** : UploadGradients (client → serveur), DownloadModel (serveur → client).

Chaque message transporte un `lamport_timestamp`. Seuils : `CHUNK_SIZE` = 4096 octets, `STREAMING_THRESHOLD` = 8192 octets. En démonstration, les gradients de 80 octets (10 floats) utilisent le mode unary ; le téléchargement du modèle emploie le streaming.

L'inscription enregistre simultanément les trois nommages :

```
self.flat_naming.register(address, flat_id=flat_id)
self.structured_naming.register(request.structured_path, address)
self.attribute_naming.register(flat_id, address, dict(request.attributes))
```

3.3 Coordination et tolérance aux pannes

Lamport (`lamport.py`) : `tick()`, `send_event()`, `receive_event(ts)` ; tie-breaker par (`time`, `process_id`).

Bully (`bully.py`) : élection découplée de gRPC via callbacks injectés (`set_send_election`, `set_send_coordinator`).

Mutex (`mutual_exclusion.py`) : `request_access()` / `release_access()` protègent `aggregate_round()` :

```

def aggregate_round(self):
    if not self.mutex.request_access("aggregation"):
        return
    try:
        self.model_weights = np.mean(all_grads, axis=0).tolist()
        self.current_round += 1
    finally:
        self.mutex.release_access("aggregation")

```

Checkpoint : SystemState persiste current_round, coordinator_id, lamport_clock, model_weights, round_history. Détection de panne par timeout heartbeat (5 s en production).

3.4 Tests et démonstration

Module	Tests	Couverture
Lamport	7	tick, send/receive, happens-before
Bully	7	élection, panne, annonce coordinateur
Nommage	12	plat, structuré, attributs
Mutex	5	accès, conflit, release
Checkpoint	7	save/load, recovery, timeout
Total	38	pytest tests/ -v

La démonstration `./run_demo.sh` lance un coordinateur (port 50051) et trois clients (`iris`, `mnist`, `cifar`) exécutant chacun 3 rounds, puis enchaîne les benchmarks.

4. Résultats expérimentaux

Les quatre expériences ont été exécutées le **2026-06-26 à 22 :50 :49** via `python experiments/benchmarks.py`. Les tableaux ci-dessous reprennent l'intégralité des métriques enregistrées dans `experiments/results/benchmark_r`

4.1 Cohérence Lamport (`lamport_round_consistency`)

Protocole : 5 processus simulés, 10 rounds; chaque processus envoie un message horodaté à tous les autres. Vérification : $C(\text{réception}) > C(\text{envoi})$ pour chaque paire send/receive.

Métrique	Valeur (JSON)
<code>num_processes</code>	5
<code>num_rounds</code>	10
<code>total_events</code>	250
<code>ordering_violations</code>	0
<code>consistent</code>	true
<code>final_clock_values</code>	[100, 100, 100, 100, 99]

Interprétation : aucune violation d’ordonnancement causal sur 250 événements. Les horloges convergent vers des valeurs cohérentes (100 pour P0–P3, 99 pour P4), confirmant le respect des règles Lamport implémentées dans `lamport.py`.

4.2 Streaming vs unary (`communication_strategies`)

Protocole : buffers de 1 024 à 262 144 octets ; comparaison unary vs découpage streaming (chunks de 4 Ko).

<code>data_size_bytes</code>	<code>num_chunks</code>	<code>unary_time_ms</code>	<code>streaming_time_ms</code>	<code>streaming_overhead_ratio</code>
1 024	1	0,0	0,004	9,83
4 096	1	0,0	0,002	7,94
16 384	4	0,0	0,003	19,63
65 536	16	0,0	0,006	30,12
262 144	64	0,0	0,035	76,30

Interprétation : le ratio d’overhead streaming croît avec la taille (9,83× à 76,30×). Unary est adapté aux messages < 8 Ko (gradients de 80 octets en démo) ; le streaming devient pertinent au-delà de 64 Ko (pipelining, robustesse mémoire).

4.3 Élection Bully (`bully_election`)

Protocole : 8 processus simulés (P0–P7) ; panne du coordinateur P7 ; élection déclenchée par P4.

Métrique	Valeur (JSON)
<code>num_processes</code>	8
<code>failed_coordinator</code>	7
<code>new_coordinator</code>	6

Métrique	Valeur (JSON)
<code>election_time_ms</code>	0,103
<code>success</code>	true

Interprétation : P4 envoie ELECTION à P5 et P6 ; P6 (ID maximal vivant) devient coordinateur en 0,103 ms. L'élection réussit (`success: true`).

4.4 Checkpoint / recovery (`fault_tolerance`)

Protocole : 9 rounds simulés ; snapshots aux rounds 3, 6 et 9 ; crash simulé (absence de heartbeat pendant 2 s).

Métrique	Valeur (JSON)
<code>rounds_before_crash</code>	9
<code>crash_detected</code>	true
<code>recovered_round</code>	9
<code>recovered_weights</code>	10 valeurs à 0,9 (modèle complet restauré)
<code>checkpoints_available</code>	<code>round_3</code> , <code>round_6</code> , <code>round_9</code>
<code>success</code>	true

Interprétation : reprise exacte au round 9 avec restauration intégrale des 10 poids du modèle. Trois snapshots disponibles ; perte maximale théorique : 2 rounds entre deux checkpoints.

4.5 Synthèse globale

Expérience (<code>experiment</code>)	Indicateur clé	Résultat
<code>lamport_round_consistency</code>	<code>ordering_violations</code>	0 / 250 événements
<code>communication_strategies</code>	ratio max. streaming	76,30× (262 Ko)
<code>bully_election</code>	<code>election_time_ms</code>	0,103 ms ; P6 élu
<code>fault_tolerance</code>	<code>recovered_round</code>	9 ; <code>success: true</code>
Tests unitaires	<code>pytest tests/ -v</code>	38/38 passés

5. Conclusion

5.1 Bilan

Ce projet met en œuvre un système d'apprentissage fédéré intégrant l'ensemble des concepts obligatoires INF4218. L'architecture en trois niveaux sépare clairement application, middleware et réseau. Chaque mécanisme distribué apporte une garantie mesurable : ordre causal (Lamport), intégrité (mutex), continuité (Bully), interopérabilité (gRPC), persistance (checkpoint). Les tests, la démonstration `./run_demo.sh` et les quatre expériences confirment le comportement attendu.

5.2 Limitations et perspectives

Limitations : entraînement ML simulé (NumPy) ; coordinateur central (goulot d'étranglement) ; Bully $O(n^2)$ en messages ; gRPC non chiffré ; clients homogènes Python ; pas de tolérance Byzantine.

Perspectives : remplacer Bully par Raft ; activer TLS/mTLS et Secure Aggregation ; clients multi-langages via gRPC ; compression de gradients ; réplication du coordinateur.

6. Références

1. Van Steen M., Tanenbaum A.S., *Distributed Systems*, 4th edition, 2024.
2. Lamport L., « Time, Clocks, and the Ordering of Events in a Distributed System », *Communications of the ACM*, 21(7), 1978.
3. Garcia-Molina H., « Elections in a Distributed Computing System », *IEEE Transactions on Computers*, 31(1), 1982.
4. Ricart G., Agrawala A.K., « An Optimal Algorithm for Mutual Exclusion in Computer Networks », *Communications of the ACM*, 24(1), 1981.
5. McMahan B. et al., « Communication-Efficient Learning of Deep Networks from Decentralized Data », *AISTATS*, 2017.
6. Documentation gRPC : <https://grpc.io/docs/languages/python/>

Reproductibilité : `pytest tests/ -v, ./run_demo.sh, python experiments/benchmarks.py`.